

PROJECT REPORT

IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

ServiceNow Incident Management

Team ID: SWTID-2026-4057

Date: 01 October 2026

1. INTRODUCTION

1.1 Project Overview

This project focuses on implementing Client Scripts and UI Policies on the ServiceNow Incident form to improve user experience, enforce field behavior, and reduce incorrect data entry. The implementation dynamically controls form fields based on user actions and incident conditions without requiring the form to be submitted.

The solution is implemented on the ServiceNow Incident [incident] table. It uses a High Impact UI Policy, UI Policy Actions, and three Client Scripts—onChange, onSubmit, and onCellEdit—to provide conditional field control, automatic updates, save-time validation, and controlled list interaction.

1.2 Purpose

The purpose is to ensure consistent and accurate Incident data entry for triage, routing, and resolution. The implementation reduces dependence on manual awareness of field rules by applying appropriate controls at the user-interface level.

- Apply dynamic field behavior when Impact is 1 - High.
- Make Assignment group mandatory for High Impact Incidents.
- Keep Urgency controlled and automatically set it to High when Impact becomes High.
- Prevent incomplete High Impact Incidents from being saved when Assigned To is empty.
- Prevent direct State changes through Incident list editing.

2. IDEATION PHASE

2.1 Problem Statement

The ideation phase identifies the user-facing problems associated with accurate and consistent ServiceNow Incident data entry.

Problem Statement	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	A ServiceNow Incident user	create and update Incident records with correct and complete information	important field behavior can depend on manual awareness	conditional controls and validation are needed to enforce Incident rules	concerned about incorrect or incomplete Incident data
PS-2	An Incident management user	handle High Impact Incidents consistently	required information and controlled fields may not be enforced automatically	UI Policies and Client Scripts can dynamically guide, validate and restrict interactions	more confident when the form enforces the required behavior

2.2 Empathy Map Canvas

User / Customer: ServiceNow Incident user who creates, updates and manages Incident records.

Says	Thinks	Does	Feels
I need the Incident form to guide me and prevent incorrect data entry.	I want the correct rules to be applied automatically when Incident conditions change.	Creates and updates Incidents, changes Impact, and works with controlled fields.	More confident when required information is enforced; concerned when incorrect data could be submitted.

Pain Points

- Incomplete or inconsistent Incident data entry.
- Dependence on manual awareness of field rules.
- Risk of saving a High Impact Incident without required information.
- Direct State editing through the Incident list.

Needs / Gains

- Conditional field control.
- Automatic field behavior.
- Save-time validation.
- Controlled list interaction.

2.3 Brainstorming

Selected Problem Statement: ServiceNow Incident users need consistent controls that dynamically guide field behavior, validate important information, and reduce incorrect data entry.

Observation / Need	Project-relevant finding
Conditional control	Make Assignment group mandatory when Impact is High.
Field behavior	Make Urgency read-only when the High Impact UI Policy is active.
Automation	Set Urgency to High when Impact changes to High.
Validation	Prevent save when Assigned To is empty for a High Impact Incident.
List interaction	Prevent direct State changes through list editing.

Group	Ideas
UI Policy Controls	High Impact Control; Assignment group mandatory; Urgency read-only; Reverse if false.
Automatic Field Behavior	onChange Client Script; automatically set Urgency to High for High Impact.
Save Validation	onSubmit Client Script; block save when Assigned To is empty for High Impact.
List Interaction	onCellEdit Client Script; block direct State updates through list editing.

Idea Prioritization

Idea	Importance	Feasibility
High Impact UI Policy – Assignment group mandatory	High	High
onSubmit Assigned To validation	High	High
Urgency read-only UI Policy Action	High	Medium
onChange automatic Urgency update	Medium	High
onCellEdit State restriction	Medium	Medium

Selected Concept: Use UI Policies and Client Scripts together to provide conditional control, automatic behavior, validation, and controlled list interaction.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User Action	System / Project Behavior	Outcome
Open Incident	Open or create Incident	Incident form is available	Ready for entry
Set Impact	Select High	UI Policy activates; Assignment group becomes mandatory; Urgency is controlled	Immediate guidance
Update	Change Impact / fields	onChange can set Urgency to High	Reduced manual work
Submit	Attempt save	onSubmit checks Assigned To for High Impact	Invalid save blocked
List Editing	Edit State in list	onCellEdit blocks direct State update	Controlled interaction
Change Impact	Change High to another value	Reverse-if-false restores applicable behavior	Dynamic form behavior
Complete	Save valid record	Incident is updated	Consistent record

3.2 Solution Requirement

The solution shall provide conditional field control, automatic field behavior, save-time validation, and controlled list interaction on the ServiceNow Incident table. The controls shall operate at the client interface so that users receive immediate guidance while creating or updating Incident records.

ID	Functional Requirement	Expected System Behavior	Implementation
FR-01	High Impact UI Policy	When Impact is 1 – High, the High Impact Control UI Policy becomes active.	UI Policy on Incident table; condition: Impact is 1 – High.
FR-02	Assignment group mandatory	Assignment group becomes mandatory when High Impact is true.	UI Policy Action / mandatory field behavior.
FR-03	Urgency read-only	Urgency becomes read-only while High Impact is active.	UI Policy Action for Urgency; Read-only = true.
FR-04	Reverse policy behavior	Policy-applied field changes are reversed when condition becomes false.	Reverse if false = true.
FR-05	Automatic Urgency update	When Impact changes to High, Urgency is set to High automatically.	onChange Client Script on Impact.
FR-06	Save validation	High Impact Incident is not saved when Assigned To is empty.	onSubmit Client Script.
FR-07	List State restriction	Direct State changes through list editing are prevented.	onCellEdit Client Script on State.

Non-Functional Requirements

- Respond dynamically to user actions on the Incident interface.
- Provide clear feedback when an action is blocked.
- Remain lightweight and focused on Incident data integrity.
- Maintain consistent behavior between applicable form conditions and user actions.

3.3 Data Flow Diagram

Data Flow: User → Incident Form → Impact / Incident Fields → UI Policy & Client Scripts → Field Control / Automatic Update / Validation → Save or Block → Incident Record.

Logical flow: The Incident user interacts with the form or list. Impact and other Incident fields are evaluated by the UI Policy and Client Scripts. The system either applies field control, performs an automatic update, validates the submission, blocks a restricted list edit, or accepts a valid Incident update.

User Stories

User Story	Acceptance Criteria
As an Incident user, I want Assignment group mandatory for High Impact.	Impact = High makes Assignment group mandatory.
As an Incident user, I want Urgency controlled for High Impact.	Urgency becomes read-only under the policy.
As an Incident user, I want Urgency automatically set for High Impact.	onChange sets Urgency to High.
As an Incident user, I want invalid High Impact submissions blocked.	onSubmit blocks save when Assigned To is empty.
As an Incident user, I want State list edits controlled.	onCellEdit blocks direct list editing.

3.4 Technology Stack

Technology / Component	Purpose in the Project	Project Usage
ServiceNow	Enterprise service management platform used to build and configure the Incident solution.	Primary development and runtime platform.
Incident Table	Stores and manages Incident records and their fields.	Target table for all UI Policies and Client Scripts.
UI Policy	Applies conditional form behavior based on field values.	High Impact Control triggered when Impact = 1 – High.
UI Policy Action	Defines field-level behavior applied by a UI Policy.	Assignment group mandatory; Urgency read-only for High Impact.
Client Script – onChange	Runs when a monitored form field changes.	Sets Urgency to High when Impact changes to High.
Client Script – onSubmit	Validates data before a form is submitted.	Prevents saving a High Impact Incident when Assigned To is empty.
Client Script – onCellEdit	Controls direct editing of records from list views.	Prevents direct State changes through Incident list editing.
JavaScript	Scripting language used for client-side ServiceNow logic.	Used in the three Client Scripts.
ServiceNow Form / List UI	User interface and interaction points.	Incident form and Incident list.

4. PROJECT DESIGN

4.1 Problem Solution Fit

Problem / Need	Solution Response	Fit
Inconsistent Incident data entry	Conditional UI control and validation	Direct
Required routing information under High Impact	Assignment group mandatory	Direct
Controlled Urgency behavior	Urgency read-only and automatically set to High	Direct
Incomplete save	onSubmit Assigned To validation	Direct
Uncontrolled State list editing	onCellEdit restriction	Direct

Summary: The solution directly addresses the stated problem by applying Incident data-entry rules at the user-interface level.

4.2 Proposed Solution

The proposed solution is to configure ServiceNow Incident form and list behavior using UI Policies and Client Scripts. The solution dynamically controls fields based on incident conditions and user actions, provides immediate validation feedback, and prevents incomplete or inappropriate updates without requiring the form to be submitted for every rule evaluation.

Component	Implementation / Behavior	Purpose
UI Policy – High Impact Control	Condition: Impact is 1 - High; Reverse if false enabled; On load enabled.	Applies high-impact field controls dynamically.
UI Policy Action – Assignment group	Assignment group is mandatory when policy condition is true.	Ensures routing information is provided.
UI Policy Action – Urgency	Urgency is read-only while Impact is 1 - High.	Prevents inappropriate manual changes.
onChange Client Script	When Impact changes to 1 - High, Urgency is set to 1 - High and an information message is shown.	Keeps urgency consistent.
onSubmit Client Script	When Impact is 1 - High and Assigned To is empty, display an error and return false.	Prevents incomplete High Impact incidents.
onCellEdit Client Script	Blocks direct State changes through list editing and asks the user to open the Incident.	Controls State updates through the form.

4.3 Solution Architecture

The architecture is a ServiceNow Incident interface control architecture. Incident users interact with the Incident form or Incident list. The ServiceNow client-side control layer evaluates field conditions and user actions through the configured UI Policy and Client Scripts.

Layer	Elements	Responsibility
User Interaction	Incident User; Incident Form; Incident List	Provides the interface through which Incident data is entered or modified.
UI Policy	High Impact Control; UI Policy Actions	Dynamically applies field behavior when Impact is 1 - High and reverses it when the condition is false.
Client Script	onChange; onSubmit; onCellEdit	Automates field values, validates save conditions, and controls list editing behavior.
Feedback	Info message; Error message; Alert	Communicates automatic actions, validation failures, and restricted operations.
Data	Impact; Assignment group; Urgency; Assigned To; State	Stores Incident values affected by the implemented controls.

Control Flow

- High Impact form flow: Impact = 1 - High → UI Policy activates → Assignment group becomes mandatory and Urgency becomes read-only → onChange sets Urgency to 1 - High.
- Save validation flow: User selects Save → onSubmit checks Impact and Assigned To → if High Impact + Assigned To empty, save is blocked and an error is shown → otherwise update is allowed.
- List edit flow: User attempts to edit State from the Incident list → onCellEdit executes → alert is displayed → callback(false) blocks the list edit.
- Reverse flow: Impact changes from High to another value → Reverse if false restores normal UI Policy behavior.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project plan organizes the implementation of Client Scripts and UI Policies for the ServiceNow Incident table into clear phases. The plan covers requirement understanding, configuration, scripting, integration, testing, validation, and final documentation.

Project Objectives

- Implement dynamic field behavior for High Impact Incidents.
- Apply mandatory and read-only controls through UI Policies.
- Automate Urgency handling when Impact changes.
- Validate required information before an Incident is saved.
- Restrict direct State changes through Incident list editing.
- Test the implemented behavior and document the final solution.

Phase	Planned Activities	Expected Output
1. Requirement Understanding	Review Incident use case, identify field-control needs, define expected user behavior.	Problem definition and requirements
2. Solution Design	Map each requirement to UI Policy or Client Script behavior; define conditions and validation rules.	Solution design and architecture
3. UI Policy Configuration	Create High Impact Control; configure Assignment group mandatory and Urgency read-only actions; enable reverse behavior.	Configured UI Policy
4. Client Script Development	Implement onChange, onSubmit, and onCellEdit Client Scripts.	Configured Client Scripts
5. Integration	Verify interaction between UI Policy and Client Scripts on the Incident form and list.	Integrated Incident controls
6. Testing & Validation	Test High Impact behavior, save validation, reverse condition, valid save, and State list-edit restriction.	Validated implementation
7. Documentation	Prepare project design, testing, and supporting documentation.	Final project documentation

Key Milestones

- Requirements and problem definition completed.
- Solution architecture and control mapping completed.
- High Impact UI Policy configured.
- onChange, onSubmit, and onCellEdit Client Scripts implemented.
- Functional scenarios validated.
- Final project documentation prepared.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The objective of performance testing for this project is to verify that the configured UI Policy and Client Scripts respond appropriately during normal Incident form and list interactions without introducing unnecessary user-interface delay. The testing focuses on the implemented client-side controls rather than server-side load or infrastructure benchmarking.

Test ID	Scenario	Action	Expected Result
PT-01	High Impact UI Policy	Set Impact to 1 - High.	Assignment group becomes mandatory and Urgency becomes read-only.
PT-02	Automatic Urgency update	Change Impact to 1 - High.	Urgency changes to 1 - High and the information message appears.
PT-03	Save validation	Attempt to save with High Impact and Assigned To empty.	Save is blocked and an error is displayed on Assigned To.
PT-04	Valid save	Provide Assigned To and save the High Impact Incident.	Record can be saved successfully.
PT-05	Reverse behavior	Change Impact from High to another value.	High Impact controls are reversed; normal field behavior returns.
PT-06	List edit restriction	Edit State directly from the Incident list.	Edit is blocked and configured alert is displayed.
PT-07	Form State update	Open Incident and update State through the form.	Form-based State update remains available.

Performance Acceptance Criteria

- Client-side controls respond during the relevant user action without requiring unnecessary form submission.
- Automatic field updates occur when the corresponding condition is triggered.
- Validation executes before an invalid save is accepted.
- Restricted list editing is stopped immediately by the onCellEdit control.
- Normal form interaction remains available when configured conditions are not active.
- No performance issue is introduced by repeated execution of the implemented client-side controls during normal use.

6.2 User Acceptance Testing

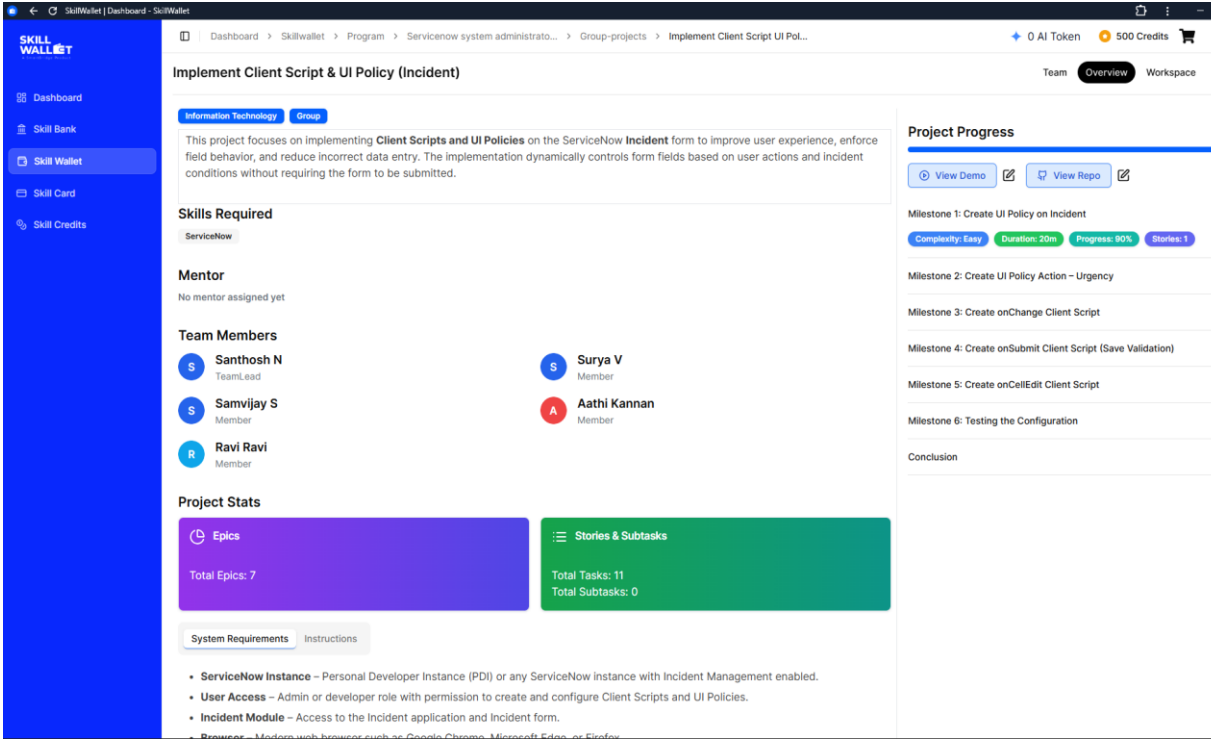
The implemented Incident controls are intended to satisfy the user-facing acceptance conditions defined by the project requirements. The UAT scenarios below verify that the expected behavior is understandable and usable from the Incident user's perspective.

UAT Scenario	Expected Acceptance
High Impact Incident	Assignment group becomes mandatory and Urgency is controlled.
Automatic Urgency	Urgency is automatically set to High when Impact changes to High.
Invalid submission	High Impact Incident with empty Assigned To cannot be saved.
Valid submission	High Impact Incident with Assigned To populated can be saved.
Reverse condition	Changing Impact away from High restores applicable normal behavior.
List State edit	Direct State editing from the list is blocked.
Form State update	State can be updated through the Incident form.

7. RESULTS

7.1 Output Screenshots

The implemented ServiceNow Incident configuration produces the expected dynamic field behavior, validation, automatic updates, and controlled list interaction. The following spaces are intentionally reserved for inserting project output screenshots.



SKILLWALLET

Dashboard

Skill Bank

Skill Wallet

Skill Card

Skill Credits

Dashboard > Skillwallet > Program > Servicenow system administrato... > Group-projects > Implement Client Script UI Pol...

0 AI Token500 CreditsSN

Implement Client Script & UI Policy (Incident)

TeamOverviewWorkspaceKanban

Milestone 2: Create UI Policy Action – Urgency

Epic

Complexity: Easy

Duration: 12h 0m

Description

Create a **UI Policy Action** for the **Urgency** field on the Incident form to control its behavior based on the conditions defined in the UI Policy. Configure the action to make the Urgency field **mandatory, visible, or read-only** as required. This ensures that the appropriate urgency information is captured consistently and helps users prioritize incidents accurately.

Resources

No resources available

Project Progress

90%

View Demo

View Repo

Milestone 1: Create UI Policy on Incident

90%

Milestone 2: Create UI Policy Action – Urgency

90%

Milestone 3: Create onChange Client Script

90%

Milestone 4: Create onSubmit Client Script (Save Validation)

90%

Milestone 5: Create onCellEdit Client Script

90%

Milestone 6: Testing the Configuration

90%

Conclusion

90%

SKILLWALLET

Dashboard

Skill Bank

Skill Wallet

Skill Card

Skill Credits

Dashboard > Skillwallet > Program > Servicenow system administrato... > Group-projects > Implement Client Script UI Pol...

0 AI Token500 CreditsSN

Implement Client Script & UI Policy (Incident)

TeamOverviewWorkspaceKanban

All Epics

Total Tasks: 11In Progress: 0Reviewed: 11Completed: 0

Switch on this button to receive your assigned tasks and have them visible on the Kanban board.☐ Assigned Task

To-Do

In Progress

To Be Reviewed

Completed

Activity 1: Create UI Policy

SN

Milestone 1: Create UI Policy on Incident

Easy

Activity 1: Configure the Urgency Field

SV

Milestone 2: Create UI Policy Action – Urgency

Easy

Activity 1: Create Client Script

SS

Milestone 3: Create onChange Client Script

Easy

Activity 1: Create Client Script

RR

Milestone 4: Create onSubmit Client Script (Save Validation)

Easy

Activity 1: Create Client Script

SS

Milestone 5: Create onCellEdit Client Script

Easy

Activity 1: Test Mandatory Enforcement

SN

Milestone 6: Testing the Configuration

Easy

Implement Client Script & UI Policy (Incident) | SWTID-2026-4057

servicenow

AllFavoritesHistoryWorkspacesIncident - Create INC0010001

Application scope: GlobalUpdate set: Default (Global)

Filter

Self-Service

- Business Applications
- Dashboards
- Service Catalog
- Employee Center
- Knowledge
- Visual Task Boards
- Incidents
- Watched Incidents
- My Requests
- Requested Items
- Watched Requested Items
- My Connected Apps
- My Profile

Channel

-- None --

State

New

Impact

1 - High

Urgency

3 - Low

Priority

3 - Moderate

* Assignment group

Assigned to

Knowledge & Catalog (All)

No results to display

servicenow

AllFavoritesHistoryWorkspaces

Client Script - New Record

Application scope: Global
Update set: Default (Global)

Submit

Client Script
New record

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

NamePrevent save if Assigned To missing

Table-- None --

UI Typeincident

TypeIncident [incident]
Incident Fact Table [incident_fact_table]
Incident Metric [incident_metric]
Incident SLA [incident_sla]
Incident Task [incident_task]
Incident Time Worked [incident_time_worked]

Description

Messages

Script

ApplicationGlobal

Active

Inherited

Global

Incidents

Number

Search

Actions on selected rows... New

	Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
	INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New		(empty)	(empty)	2018-12-12 23:30:24	admin
	INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)	(empty)	2018-12-12 23:18:55	admin
	INC0009004	2018-09-01 06:13:30	Defect tracking tool is down.	David Miller	3 - Moderate	In Progress	Software	(empty)	(empty)	2026-04-30 01:41:55	system
	INC0009003	2018-08-30 02:17:32	Cannot sign into the company portal app	David Miller	3 - Moderate	On Hold	Inquiry / Help	(empty)	(empty)	2018-12-12 23:39:53	admin
	INC0009002	2018-09-16 05:49:23	My computer is not detecting the headphone device	David Miller	3 - Moderate	Resolved	Hardware	(empty)	(empty)	2026-04-30 01:41:54	system
	INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	Closed	Inquiry / Help	(empty)	(empty)	2018-12-12 23:32:42	admin
	INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	5 - Planning	Canceled	Inquiry / Help	(empty)	(empty)	2019-07-29 11:49:28	admin
	INC0008111	2019-07-22 14:04:57	ATF : Test1	System Administrator	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-22 14:05:48	admin
	INC0008001	2021-01-15 13:04:14	ATF:TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2021-01-21 15:31:42	admin
	INC0007002	2018-10-16 22:47:51	Need access to the common drive.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:28:49	admin
	INC0007001	2018-10-16 22:47:51	Employee payroll application server is down	David Miller	1 - Critical	New	Hardware	Openspace	(empty)	2026-04-30 01:41:54	system

1 to 20 of 67

8. ADVANTAGES & DISADVANTAGES

Advantages

- Improves consistency of Incident data entry.
- Provides immediate user guidance at the point of interaction.
- Reduces manual checking of conditional field requirements.
- Automatically aligns Urgency with High Impact selection.
- Prevents incomplete High Impact Incidents from being saved.
- Controls direct State editing from the Incident list.
- Uses native ServiceNow UI Policy and Client Script mechanisms.
- Reverse-if-false behavior allows normal form behavior when the condition is no longer active.

Disadvantages

- The solution is specific to the configured ServiceNow Incident table and field behavior.
- Client-side controls depend on correct configuration of UI Policies and Client Scripts.
- The implementation does not by itself provide a server-side enforcement layer for every possible data-entry path.
- Additional rules or Incident fields would require further configuration and testing.
- Large-scale infrastructure or server-load performance is outside the scope of this client-side implementation.

9. CONCLUSION

The project successfully defines and implements a focused ServiceNow Incident interface-control solution using UI Policies and Client Scripts. The High Impact Control UI Policy applies mandatory and read-only behavior, the onChange Client Script automates Urgency handling, the onSubmit Client Script validates required information before saving, and the onCellEdit Client Script restricts direct State changes through list editing.

The consolidated design, requirement, planning, and testing activities show a direct relationship between the identified Incident data-entry problems and the implemented controls. The solution provides immediate user feedback and supports more consistent Incident handling while retaining normal behavior when the configured conditions are not active.

10. FUTURE SCOPE

The current implementation provides the required Incident-level client-side controls. The following areas can be considered for future enhancement:

- Extend conditional controls to additional Incident fields and business conditions.
- Add more comprehensive server-side validation where stronger enforcement is required.
- Introduce role-based behavior for specialized Incident management users.
- Add additional automated routing or assignment rules based on Incident category, service, or priority.
- Expand user feedback with richer guidance for different Incident conditions.
- Introduce broader automated test coverage for Client Scripts and UI Policies.
- Add analytics to measure validation failures, repeated corrections, and Incident data-quality improvements.
- Integrate additional notification or workflow automation where required by the Incident management process.

11. APPENDIX

Source Code (if any)

The project uses ServiceNow Client Scripts and UI Policy configuration. The implemented script/configuration names are listed below for reference:

Component	Name / Configuration
UI Policy	High Impact Control
UI Policy Action	Assignment group – Mandatory
UI Policy Action	Urgency – Read-only
onChange Client Script	Auto set urgency for high impact
onSubmit Client Script	Prevent save if Assigned To missing
onCellEdit Client Script	Prevent state change via list edit

Dataset Link

Not provided / Not applicable. The project is a ServiceNow Incident configuration and does not depend on an external machine-learning dataset.

GitHub & Project Demo Link

Github Link :

<https://github.com/sandytechie0806/Implement-Client-Script-UI-Policy-Incident-.git>

Demo Link :

<https://drive.google.com/file/d/1wjYrc1Mw4SQ5KYWK2g4v5YTzmYsXLLwf/view?usp=drivesdk>

Project Identification

Item	Details
Project Name	Implement Client Script & UI Policy (Incident)
Team ID	SWTID-2026-4057
Platform	ServiceNow
Target Table	Incident [incident]
Date	01 October 2026